



Лабораторная работа № 3

Вторая работа может выпить душу из некоторых студентов.

Но теперь точно бояться нечего!

Dependency Injection (внедрение зависимостей)

Необходимо добиться, чтобы объект, реализующий `IRepository`, внедрялся в объект класса `BusinessLogic` средствами IoC-контейнера (например `Ninject`) и представлял из себя Singleton.

Итак, нужно доработать лабораторную работу №2 следующим образом:

- ✓ В слое View создавать объект класса `BusinessLogic` необходимо через IoC-контейнер с целью автоматического внедрения зависимого объекта (репозитория) в объект BL.
- ✓ В слое `BusinessLogic` создаем сам контейнер.

Пояснения:

1. Во второй лабораторной работе класс Бизнес-логики содержит ссылку на конкретную реализацию репозитория (`EntityRepository` или `DapperRepository`). Для того, чтобы убрать жесткую связь между двумя классами (классом бизнес-логики и репозитория) первым шагом определяем интерфейс `IRepository` и в классе бизнес-логики определяем конструктор с параметрами:

```
public class Logic
{
    public IRepository Repository { set; get; }

    public Logic(IRepository repository)
    {
        Repository = repository;
    }
}
```

Таким образом мы объявили зависимость бизнес-логики от объекта `IRepository` (а не от конкретного класса репозитория, как было раньше).

2. Установим пакет `Ninject` через управление пакетами `NuGet`
3. Зарегистрируем зависимости для всех репозиториях. Для этого создадим в библиотеке бизнес-логики новый класс `SimpleConfigModule`:

```
public class SimpleConfigModule : NinjectModule
{
    public override void Load()
    {
        Bind<IRepository<Student>>().To<EntityRepository<Student>>().
            InSingletonScope();
    }
}
```

Этим шагом мы устанавливаем сопоставление между интерфейсом-зависимостью и конкретным классом этого интерфейса.

4. На уровне представления (там, где мы ранее создавали экземпляр класса бизнес-логики), для управления зависимостями через Ninject создадим объект `Ninject.IKernel` с помощью встроенной реализации этого интерфейса - класса `StandardKernel`:

```
IKernel ninjectKernel = new StandardKernel(new SimpleConfigModule());
```

В этом случае создания экземпляра класса бизнес-логики необходимо реализовать так:

```
Logic bL = ninjectKernel.Get<Logic>();
```